40 Days of LeetCode Challenge

40 days, 40 problems
DAY :6

LEETCODE PROBLEM NO. 412 : Fizz Buzz

Problem Link : https://leetcode.com/problems/fizz-buzz/
Easy · Nested Ternary · arithmetic operators · Loop
STEP BY STEP SOLUTION(The following problem can be solved using 2 efficient & optimized methods) :-
note :- Apart from this solution there are many possible approaches as well

Step 1. Understand the question :-

We are given an integer n. We need to create and return a list of strings containing every number from 1 to n, but so me numbers must be replaced with special words. The rules are:

- If a number is divisible by both 3 and 5, add "FizzBuzz".
- If it's divisible only by 3, add "Fizz".
- If it's divisible only by 5, add "Buzz".
- Otherwise, add the number itself as a string.

NOTE :-

The answer starts at 1, not 0, and every item in the returned list is a String, including ordinary numbers like "1"and "2".

Step 2. Translating it into simpler language with examples :-

Example 1:
Input: n = 3
Output: ["1","2","Fizz"]
Example 2:
Input: n = 5
Output: ["1","2","Fizz","4","Buzz"]
Example 3:
Input: n = 15
Output: ["1","2","Fizz","4","Buzz","Fizz","7","8","Fizz","Buzz","11","Fizz","13","14","FizzBuzz"]
Imagine we're counting from 1 to 15

- Multiples of 3 say Fizz.
- Multiples of 5 say Buzz.
- Multiples of both say FizzBuzz.
- Otherwise, the number itself

For example:

Number	Why?	Output
1	Neither 3 nor 5 divides it	"1"
3	Divisible by 3	"Fizz"
5	Divisible by 5	"Buzz"
9	Divisible by 3	"Fizz"
10	Divisible by 5	"Buzz"
15	Divisible by both 3 and 5	"FizzBuzz"

Step 3. Identify the pattern

This problem uses three fundamental programming concepts.

A. Loops

We need to process every integer from 1 to n, so for loop it is the user's choice.

B. The modulo operator %

The modulo operator gives the remainder after division.

- num % 3 == 0 means num is divisible by 3.
- num % 5 == 0 means num is divisible by 5.
- num % 15 == 0 means num is divisible by both 3 and 5, because (3 x 5 = 15).

For example, 15 % 3 gives 0, and 15 % 5 also gives 0.

C. If-else conditions

We use conditional statements to decide which string belongs at each position.

OR

C. Ternary operator (?:)

The ternary operator is a compact way to select one of two values depending on a condition.

Its syntax is:

condition ? valueIfTrue : valueIfFalse;

D. Nested ternary operators

A nested ternary places another ternary expression inside the false branch of the previous one. This lets us test several conditions in sequence.

NOTE :

condition order matters. We check divisibility by 15 first so that 15 becomes "FizzBuzz", rather than bein g caught by the condition for "Fizz".

NOTE:-

check the most specific condition first. If we check divisibility by 3 first, the number 15 would become "Fizz" instead of "FizzBuzz".

Step 4. Build the logic

Here's the approach we'll follow:

1. Create an empty List<String> to store the answers.
2. Start a for loop at 1 and continue until n, including n.
3. For each number, check whether it is divisible by both 3 and 5. If yes, add "FizzBuzz".

4. Otherwise, check divisibility by 3 and add "Fizz".
5. Otherwise, check divisibility by 5 and add "Buzz".
6. If none of those conditions is true, convert the number to a string and add it.
7. Return the completed list.

OR

1. Create an empty List<String> named answer.
2. Use a for loop to iterate from 1 through n.
3. Check whether the current number is divisible by 15.
4. If it isn't, check whether it's divisible by 3.
5. If that condition also fails, check whether it's divisible by 5.
6. If none of these conditions match, convert the number into a string.
7. Store the selected string in a variable called value.
8. Add value to answer.
9. Return answer after the loop finishes.

Algorithm

1. Create an empty list called answer.
2. For num from 1 to n:
   a. If num is divisible by both 3 and 5:
      Add "FizzBuzz".
   b. Else if num is divisible by 3:
      Add "Fizz".
   c. Else if num is divisible by 5:
      Add "Buzz".
   d. Else:
      Add num as a string.
3. Return answer.
4. Stop.

OR

1. Create an empty list named answer.
2. For each number i from 1 to n:
   a. If i is divisible by 15, value = "FizzBuzz".
   b. Otherwise, if i is divisible by 3, value = "Fizz".
   c. Otherwise, if i is divisible by 5, value = "Buzz".
   d. Otherwise, value = the string representation of i.
   e. Add value to answer.
3. Return answer.

Pseudocode

```
ALGORITHM FizzBuzz(n)

BEGIN

  answer← empty list

  FOR num ← 1 TO n DO
```

```
    IF num MOD 3 = 0 AND num MOD 5 = 0 THEN
      ADD "FizzBuzz" TO answer
    ELSE IF num MOD 3 = 0 THEN
      ADD "Fizz" TO result
    ELSE IF num MOD 5 = 0 THEN
      ADD "Buzz" TO answer
    ELSE
      ADD STRING(num) TO answer
    END IF
  END FOR
  RETURN answer
END
```

NOTE:-
Notice that the pseudocode uses MOD to represent Java's % operator and STRING(num) to represent converting a n integer into a string.

OR

```
ALGORITHM FizzBuzz(n)
BEGIN
  answer ← empty list

  FOR i ← 1 TO n DO
    value ←
      IF i MOD 15 = 0 THEN "FizzBuzz"
      ELSE IF i MOD 3 = 0 THEN "Fizz"
      ELSE IF i MOD 5 = 0 THEN "Buzz"
      ELSE STRING(i)

    ADD value TO answer
  END FOR

  RETURN answer
END
```

Step 5. Code in Java

This is the standard arithmetic solution for LeetCode 412.

```
import java.util.ArrayList;
import java.util.List;

class Solution {
```

```
   public List<String> fizzBuzz(int n) {
      List<String> answer = new ArrayList<>();
      for (int i = 1; i <= n; i++) {
         if (i % 3 == 0 && i % 5 == 0) {
            answer.add("FizzBuzz"); }

          else if (i % 3 == 0) {
           answer.add("Fizz"); }

          else if (i % 5 == 0) {
           answer.add("Buzz"); }

          else {
           answer.add(String.valueOf(i); }
      } return answer;
   }
}
```

NOTE:-

Why this solution?

- It checks every number exactly once.
- It stops checking a number as soon as a matching condition is found.
- It uses a single loop and a single result list.
- It avoids extra helper methods and unnecessary variables.

OR

```
import java.util.ArrayList;
import java.util.List;
class Solution {
   public List<String> fizzBuzz(int n) {
      List<String> answer = new ArrayList<>();

      for (int i = 1; i <= n; i++) {
         String value = (i % 15 == 0) ? "FizzBuzz"
                  : (i % 3 == 0)  ? "Fizz"
                  : (i % 5 == 0)  ? "Buzz"
                  : String.valueOf(i);
         answer.add(value);
      } return answer;
   }
}
```

How to submit

1. Open LeetCode Problem 412. Fizz Buzz
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

A. Import the required classes

```
import java.util.ArrayList;
import java.util.List;
```

- List is an interface that represents an ordered collection.
- ArrayList is a class that implements List and lets us store elements dynamically.

B. Define the method

```
class Solution {
   public List<String> fizzBuzz(int n) {
```

- class Solution is the class LeetCode expects.
- public means the method can be accessed by the testing system.
- List<String> means the method returns a list whose elements are strings.
- fizzBuzz(int n) accepts the integer n.

C. Line 5: Create the result list

```
List<String> answer = new ArrayList<>();
```

Creates an initially empty list. We'll add each answer to it as we process the numbers.

D. Iterate from 1 through n

```
for (int i = 1; i <= n; i++) {
```

- int i = 1: start counting at 1.
- i <= n: continue until we process n.
- i++: increase the number by 1 after each iteration.

E. Check both divisibility conditions first

```
if (i % 3 == 0 && i % 5 == 0) {
   answer.add("FizzBuzz");
}
```

The && operator means both conditions must be true. If i is divisible by both 3 and 5, we add "FizzBuzz".

F. Check divisibility by 3

```
else if (i % 3 == 0) {
   answer.add("Fizz");
}
```

This runs only if the first condition was false. If the number is divisible by 3, we add "Fizz".

G. Check divisibility by 5

```
else if (i % 5 == 0) {
   answer.add("Buzz");
}
```

If the previous conditions failed but the number is divisible by 5, we add "Buzz".

H. Handle every other number

```
else {
   answer.add(String.valueOf(i));
}
```

String.valueOf(i) converts the integer into a string.

For example, the integer 7 becomes the string "7". This conversion matters because the method must return List<String>, not a mixture of integers and strings.

I. Return the result

return answer;

Once the loop has processed every number, we return the completed list.

Dry run: n = 5

Iteration (i)	Condition matched	Added to list
1	None	"1"
2	None	"2"
3	Divisible by 3	"Fizz"
4	None	"4"
5	Divisible by 5	"Buzz"

Final output

["1", "2", "Fizz", "4", "Buzz"]

Dry run: n = 15

When i = 15:

1. 15 % 3 == 0 → true.
2. 15 % 5 == 0 → true.
3. Both conditions are true, so the first if adds "FizzBuzz".
4. The remaining else if and else branches are skipped

OR

A. Import the collection classes

import java.util.ArrayList;
import java.util.List;

- List represents an ordered collection of elements.
- ArrayList is the implementation we use to store our results.

B. Define the class and method

class Solution {
public List<String> fizzBuzz(int n) {

- Solution is the class LeetCode uses for the submitted code.
- public makes the method accessible to the testing system.
- List<String> indicates that the return value is a list of strings.
- fizzBuzz(int n) accepts the integer n.

C. Create the answer list

List<String> answer = new ArrayList<>();

Creates an empty list that will hold the output.

D. Loop through all numbers

for (int i = 1; i <= n; i++) {

- int i = 1: start at 1.
- i <= n: include n in the loop.
- i++: increase i by one after each iteration.

E. Choose a string using nested ternary operators

```
String value = (i % 15 == 0) ? "FizzBuzz"
        : (i % 3 == 0)  ? "Fizz"
        : (i % 5 == 0)  ? "Buzz"
        : String.valueOf(i);
```

Let's break this expression down into individual decisions.

First condition: (i % 15 == 0) ? "FizzBuzz"

Checks whether i is divisible by 15. If true, the chosen value is "FizzBuzz".

Second condition: (i % 3 == 0) ? "Fizz"

introduces the alternative when the previous condition is false. If i is divisible by 3, the chosen value is "Fizz".

Third condition: (i % 5 == 0) ? "Buzz"

NOTE:-

Each **?** separates a condition from the value chosen when it is true.

Each **:** introduces what happens when that condition is false.

Why is String.valueOf(i) necessary?

Because i is an integer, but answer stores strings. This method converts, for example, the integer 8 into the string "8".

F. Add the selected value

answer.add(value);

After choosing the correct string, we add it to the list.

G. Return the list

return answer;

Once all numbers have been processed, return the completed answer.

Dry run A: n = 5

i	Divisible by 15?	Divisible by 3?	Divisible by 5?	Value added
1	No	No	No	"1"
2	No	No	No	"2"
3	No	Yes	Not checked	"Fizz"
4	No	No	No	"4"
5	No	No	Yes	"Buzz"

Final output: ["1", "2", "Fizz", "4", "Buzz"]

Dry run B: n = 15

```
String value = (15 % 15 == 0) ? "FizzBuzz"
        : (15 % 3 == 0)  ? "Fizz"
        : (15 % 5 == 0)  ? "Buzz"
        : String.valueOf(15);
```

1. 15 % 15 == 0 evaluates to true.
2. The expression selects "FizzBuzz".
3. The remaining conditions aren't evaluated.
4. answer.add(value) adds "FizzBuzz" to the list.

The final output is: ["1", "2", "Fizz", "4", "Buzz", "Fizz", "7", "8", "Fizz", "Buzz", "11", "Fizz", "13", "14", "FizzBuzz"]

NOTE:-

Step 6. Analyze complexity and optimization

Let (n) be the input number.

Time complexity: (O(n))

We iterate from 1 through n. Each number requires a constant number of divisibility checks, so the total running time grows linearly with n.

Auxiliary space complexity: (O(1))

The loop uses only a constant amount of additional working memory.

However, the output list itself requires (O(n)) space, because it stores n strings. Therefore, the total space used, including the returned result, is (O(n)).

NOTE :-

Can we optimize it further?

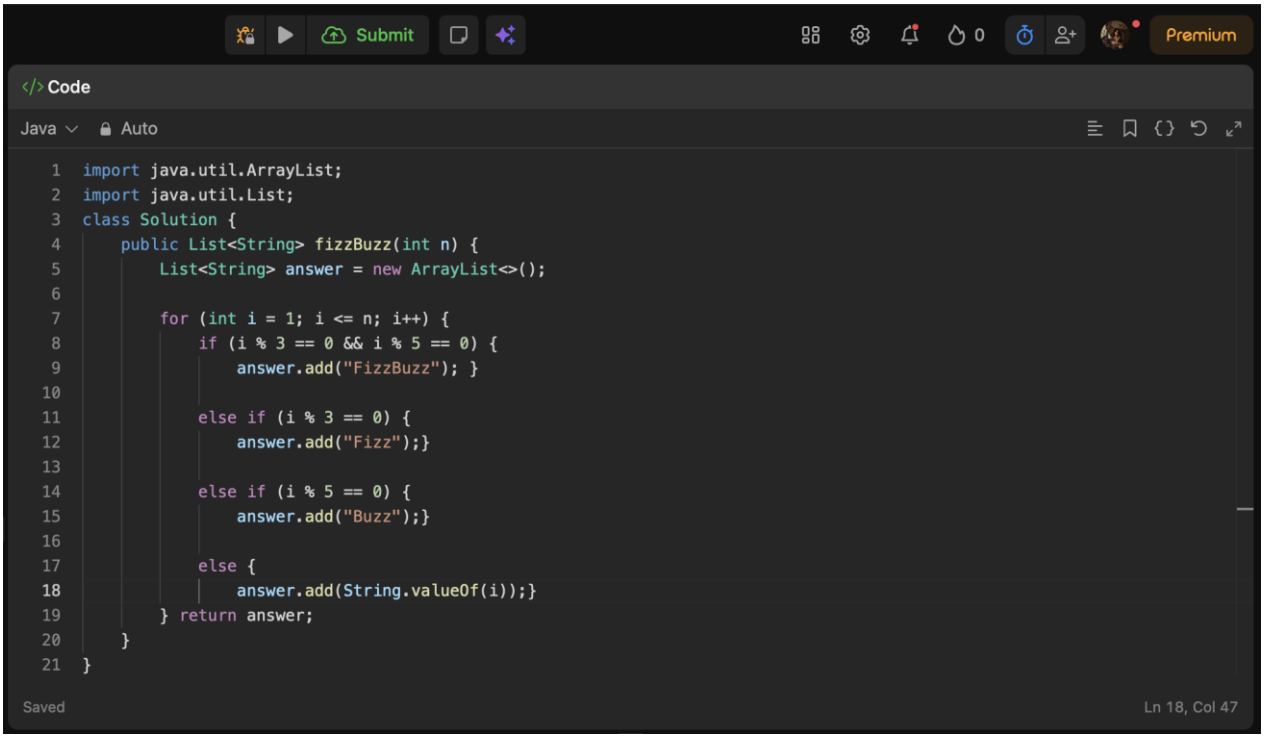
Our solution already achieves the expected optimal **time complexity**: (O(n)), because every number must be processed and every answer must be returned.

Is the ternary version more efficient than if-else?

Not necessarily Both approaches have (O(n)) **time complexity** and (O(1)) **auxiliary space**.

The ternary version is a useful exercise in Java syntax, but it doesn't guarantee a faster runtime. Its main advantage here is practising conditional expressions in a compact form.

SCREENSHOT:-



OR

